CASE STUDY REPORT

TeleLens: Telecom Feedback Intelligence

Word2Vec, Sentiment Classification, Text Summarization and Factoid Question Answering in One Evaluated System

---

Artificial Intelligence / Natural Language Processing

Prepared by:	Bharat Chandra Bodapati
Student ID:	2023005380
Study domain:	Telecom customer feedback
Primary application:	Complaint triage, opinion digests and evidence-backed answers
Report date:	1 November 2026
Source code:	github.com/BharatChandra-sys/NLP-case-study
Submitted to:	Radha Karampudi, Assistant Professor Department of Computer Science and Systems Engineering GITAM School of Computer Science and Engineering, Hyderabad

Research integrity note. Every score in this report is generated from the result files in the linked repository, produced by notebooks executed on a Kaggle T4 GPU. Word pairs, questions and thresholds used for evaluation are separated from those used for fitting, and every change made after seeing test results is disclosed in Section 10. Results that were neutral or negative are reported alongside the positive ones. The factoid gold set is a single-annotator set and has not been independently validated.

Contents

Executive Summary **1**

1 Problem Statement **1**

1.1 Background and real-world need . . . 1
1.2 Problem statement . . . 1
1.3 Why the problem is hard . . . 1
1.4 Objectives . . . 2

2 Data and Experimental Protocol **2**

2.1 Datasets . . . 2
2.2 Evaluation protocol and leakage control . . . 2
2.3 Metrics and why each is used . . . 3

3 System Architecture **3**

4 Case Study 1: Word2Vec **4**

4.1 Theory . . . 4
4.2 Original contribution: Polarity-Aware Word2Vec (PA-W2V) . . . 5
4.3 Implementation and logic . . . 5
4.4 Results . . . 7

5 Case Study 2: Sentiment Classification **10**

5.1 Methods . . . 10
5.2 Implementation and logic . . . 10
5.3 Results . . . 11
5.4 Original contribution: selective triage . . . 13
5.5 Robustness . . . 14

6 Case Study 3: Text Summarization **14**

6.1 Original contribution: PAB-MMR . . . 15
6.2 Implementation and logic . . . 15
6.3 Recovering sentences from unpunctuated text . . . 17
6.4 Evaluation against human references . . . 17
6.5 Evaluation on telecom digests . . . 18

7 Case Study 4: Factoid Question Answering **20**

7.1 Pipeline . . . 20
7.2 Implementation and logic . . . 21
7.3 Why a new gold set was needed . . . 22
7.4 Public benchmark . . . 22
7.5 Retrieval . . . 23

7.6 End-to-end results 23
7.7 Error analysis 25

8 Integrated System **25**

9 Comparison with Supplied References **26**

10 Threats to Validity and Disclosed Limitations **27**

11 Conclusion and Future Work **27**

References **28**

A Evidence Index **30**

B Execution Record **30**

C Reproducibility **30**

Executive Summary

This report extends an earlier telecom feedback triage prototype, which measured a TF–IDF classifier and described Word2Vec, into a single system covering all four assigned topics. Each topic is treated as a full case study with theory, implementation, a measured baseline comparison and an original contribution. The four contributions share one theme: *a language system should represent opposite meanings as different, preserve the balance of what customers said, and decline to decide when the evidence is weak.*

Table 1: Assignment coverage and headline evidence.

Assigned topic	Original contribution	Headline measured result	Evidence
Case Study on Word2Vec (Sec. 4)	PA-W2V: negation binding and polarity counter-fitting with labels nested in folds	Held-out antonym cosine 0.683 → 0.373; neighbour purity 0.773 → 0.839	`Notebook 01`
Case Study on Sentiment Classification (Sec. 5)	Calibrated selective triage with cross-fitted deferral thresholds	Macro-F1 0.916 (n = 9,874); 5% risk target met at 4.93% with 91% coverage	`Notebook 02`
Case Study on Text Summarization (Sec. 6)	PAB-MMR: polarity- and aspect-balanced selection; punctuation recovery for unpunctuated text	Polarity gap 0.059 → 0.019; aspect recall 0.680 → 0.850; ROUGE statistically tied	`Notebook 03`
Case Study on Factoid QA (Sec. 7)	Answer-type gating and calibrated abstention over complaints	Overall accuracy 0.477 → 0.602; answered EM 0.755; 80% of unanswerable rejected	`Notebook 04`

1 Problem Statement

1.1 Background and real-world need

Telecom providers receive short, informal feedback about broadband reliability, mobile coverage, billing, installation and support. Support teams must identify negative experiences quickly, understand what customers are saying in aggregate, and look up specific facts inside historical complaints. These are four different language tasks, and in practice they are usually built as disconnected tools. This project builds them as one system, **TeleLens**, over real public telecom reviews.

1.2 Problem statement

How can one NLP system use distributional word representations to classify telecom feedback, summarize it without distorting its sentiment, and answer factual questions with evidence, while measuring its failures and declining to decide when its evidence is weak?

1.3 Why the problem is hard

- **Antonyms share contexts.** "Stable" and "unstable" occur in near-identical sentences, so Word2Vec places them close together. The earlier report showed TF–IDF cosine of 0.872 between "the service is reliable" and "the service is not reliable".
- **Summaries can distort.** A summary that selects central sentences can silently turn a mixed set of reviews into an all-negative or all-positive one.
- **The main corpus has no punctuation or digits.** Sentence-based summarization cannot be applied directly, and numeric facts have been erased.
- **Wrong answers are costly.** A question answering system that always answers will invent a fact when the corpus does not contain one.

1.4 Objectives

1. Implement skip-gram Word2Vec from scratch, validate it against library implementations and improve its handling of opposite meanings.
2. Compare sentiment models under leakage-controlled evaluation and turn the best into a calibrated triage policy.
3. Build a summarizer that preserves sentiment balance and aspect coverage, and evaluate it against human references.
4. Build a factoid question answering pipeline with evidence and abstention, and evaluate it on a public benchmark and a domain gold set.
5. Integrate all four into a single working system and publish code, notebooks and results.

2 Data and Experimental Protocol

2.1 Datasets

Table 2: Datasets, role and provenance.

Dataset	Size used	Role and source
WEDataset `Data10KMixed`	9,874 reviews after removing blank rows and 13 conflicting-label groups; 617,573 tokens	Word2Vec, sentiment, telecom digests. HeshamAsem/WEDataset, linked from reference [1]
WEDataset `Data1KMixed`	999 reviews	Continuity with the earlier report
Comcast ConsumerAffairs	5,629 complaints; 34,054 passages	Factoid QA corpus; cross-domain sentiment test. Kaggle
FewSum Amazon gold	60 products, 8 reviews each, 3 human summaries	Summarization references: 40 dev, 20 test. abrazinskas/FewSum
SQuAD v1.1 dev	Seeded sample of 2,000 questions	Public reader benchmark. SQuAD
SimLex-999	999 pairs; about 250 in vocabulary	External similarity benchmark. SimLex-999
TeleLens factoid gold	102 answerable, 30 unanswerable; 44 dev, 88 test	Domain QA evaluation. `data/gold/qa_gold.jsonl`

The WEDataset repository declares no licence, so the raw file is downloaded at run time and is not redistributed in the project repository. The 1,000-row and 10,000-row files share only part of their text (15.4% of the 1K reviews occur in the 10K file) and must not be confused with the 367,000-review corpus described in reference [1].

2.2 Evaluation protocol and leakage control

- **Grouped five-fold validation.** Identical review texts stay in one fold (`StratifiedGroupKFold`, seed 42).
- **Fold-nested fitting.** Vocabulary, IDF, embeddings, polarity scores and classifier calibration are fitted on the training fold only.
- **Held-out word pairs.** Word pairs used to evaluate embeddings are excluded from counter-fitting constraints.
- **Cross-fitted thresholds.** A deferral threshold applied to one fold is chosen from the other four.
- **Development and test separation.** Summarizer weights, QA configuration and the abstention threshold are selected on development data only.

- **Paired statistics.** Bootstrap confidence intervals, exact McNemar tests and Wilcoxon signed-rank tests support each comparison.

2.3 Metrics and why each is used

No single number describes a language system, so each task is measured on several complementary metrics.

Table 3: Metrics used in this report.

Metric	Meaning	Used for
Macro-F1, accuracy	Class-balanced harmonic mean of precision and recall; share correct	Sentiment
ROC-AUC, PR-AUC	Ranking quality across all thresholds	Sentiment, answerability
MCC, Cohen's κ	Agreement corrected for chance; MCC uses all four confusion cells	Sentiment
Brier, log loss, ECE	Quality of probabilities and gap between confidence and accuracy	Calibration
Risk and coverage	Error on handled items and share handled	Selective triage, abstention
Hit@k	Whether a correct item appears in the top k results	Retrieval, answers, triage queue, word neighbours
MRR, nDCG@10	Reward for placing the correct item high in the ranking	Retrieval
Exact match, token F1	Standard SQuAD answer scores	Factoid QA
ROUGE P/R/F	Word and bigram overlap with human references	Summarization
Polarity gap, sentiment hit	JS divergence from true sentiment share; agreement on majority sentiment	Summary fidelity
Aspect recall, redundancy	Topic coverage; mean similarity between selected sentences	Summary quality
Boundary P/R/F	Recovery of true sentence boundaries	Segmentation
Cosine, AUC, purity, SimLex ρ	Word-pair similarity, synonym–antonym separation, neighbour polarity	Word2Vec

For a question with one relevant document at rank r_q over a question set Q, the ranking metrics are

$$\text{Hit@}k = \frac{1}{|Q|}\sum_q \mathbf{1}[r_q \le k], \quad \text{MRR} = \frac{1}{|Q|}\sum_q \frac{1}{r_q}, \quad \text{nDCG@10} = \frac{1}{|Q|}\sum_q \frac{\mathbf{1}[r_q \le 10]}{\log_2(r_q+1)}, \tag{1}$$

and the Matthews correlation coefficient and Brier score are

$$\text{MCC} = \frac{TP\cdot TN - FP\cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}, \qquad \text{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2. \tag{2}$$

3 System Architecture

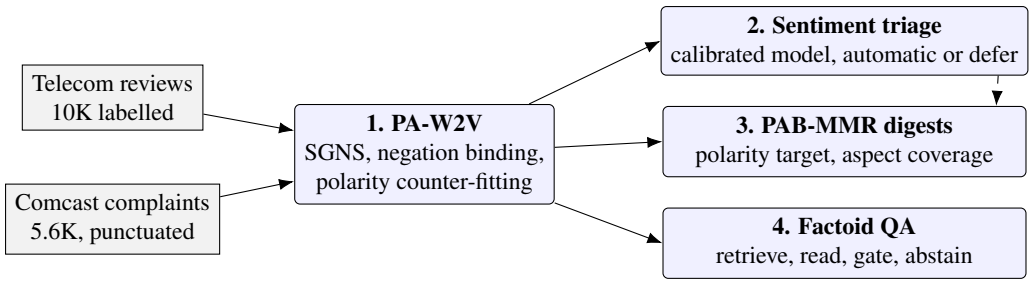


Figure 1: TeleLens architecture. Embeddings feed the downstream tasks; sentence polarity from the sentiment model feeds the summarizer (dashed).

Core logic is in `src/telelens`, one module per task. Notebooks in `notebooks` run the experiments, and `pipeline.py` exposes three operations: `triage`, `digest` and `ask`. Heavy computation ran on Kaggle (Tesla T4 GPU); the local machine holds code and pulled outputs. The code listings below are extracted automatically from the repository source by `make_code_listings.py`, so they are exactly the code that ran.

Table 4: Implementation settings.

Component	Settings
SGNS	200 dimensions, window 5, 10 negatives, 5 epochs, minimum count 5, subsampling 10^{-3}, SparseAdam at 0.002, batch 8,192
Counter-fitting	$\lvert z\rvert \geq 2.58$, 10 neighbours, repel weight 1, attract weight 0.3, anchor weight 1, attract margin 0.6, 300 Adam steps at 0.01
TF–IDF classifier	Unigrams and bigrams, minimum document frequency 2, 50,000 features, sublinear TF, logistic regression $C = 4$, Platt scaling
DistilBERT	`distilbert-base-uncased`, 2 epochs, batch 32, learning rate 2×10^{-5}, 256 tokens, temperature scaling
PAB-MMR	50-word budget; tuned $\lambda = 0.9, \beta = 0, \eta = 0.5$; telecom $\lambda = 0.7, \beta = 0.3, \eta = 0.5$
Punctuation model	`oliverguhr/fullstop-punctuation-multilang-large`, 200-word windows
BM25 and retrieval	$k_1 = 1.5$, $b = 0.75$; passages of 3 sentences with stride 2; reranking weight 0.25 on BM25's top 100
Reader and abstention	`deepset/roberta-base-squad2`, 384 tokens, top 10 passages, retrieval bonus $w = 2$, $\tau = 3.27$

4 Case Study 1: Word2Vec

Abstract. Word2Vec represents a word by the contexts it occurs in, which also places opposites such as *stable* and *unstable* close together. This case study implements skip-gram with negative sampling from scratch and introduces Polarity-Aware Word2Vec (PA-W2V), which adds negation binding and polarity counter-fitting learned only from training-fold labels. On 9,874 telecom reviews under grouped five-fold validation, with evaluation word pairs excluded from fitting, mean antonym cosine fell from 0.683 to 0.373, antonyms among the ten nearest neighbours fell from 0.087 to 0.013, and neighbour polarity purity rose from 0.773 to 0.839. Embedding-based sentiment macro-F1 rose from 0.871 to 0.885. The gain has costs: synonym Hit@10 fell from 0.587 to 0.560, SimLex-999 did not improve, and mean pooling hides much of the effect at sentence level.

4.1 Theory

Skip-gram with negative sampling (SGNS) [5,6] learns a target vector $\mathbf{v}_w$ and context vector $\mathbf{u}_c$. For an observed pair (w, c) and K noise words drawn from $P_n(w) \propto \text{count}(w)^{3/4}$ it minimises

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) - \sum_{i=1}^{K} \log \sigma(-\mathbf{u}_{n_i}^\top \mathbf{v}_w). \tag{3}$$

Using $\sigma'(x) = \sigma(x)\sigma(-x)$ gives the gradient

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_w} = -\sigma(-\mathbf{u}_c^\top \mathbf{v}_w)\,\mathbf{u}_c + \sum_{i=1}^{K} \sigma(\mathbf{u}_{n_i}^\top \mathbf{v}_w)\,\mathbf{u}_{n_i}, \tag{4}$$

which pulls $\mathbf{v}_w$ towards its observed contexts and pushes it from noise words. This explains the weakness studied here: if two words share contexts, the same $\mathbf{u}_c$ pull them to the same place, *whether they are synonyms or antonyms*. FastText [7] adds character n-grams but does not remove the effect and, because "stable" and "unstable" share character n-grams, can worsen it.

4.2 Original contribution: Polarity-Aware Word2Vec (PA-W2V)

PA-W2V adds two steps around SGNS, requiring no external lexicon.

Step 1: negation binding. Before training, a negator and the next content word become one token: "not very reliable" becomes `not_reliable`. This extends the negation marking of Pang et al. [11] to embedding training and gives negated meaning its own vector.

Step 2: polarity counter-fitting. Word polarity is estimated by a log-odds ratio with an informative Dirichlet prior [10], using only training-fold labels:

$$z_w = \frac{\log \frac{y_w^+ + a}{n^+ + a_0 - y_w^+ - a} - \log \frac{y_w^- + a}{n^- + a_0 - y_w^- - a}}{\sqrt{\frac{1}{y_w^+ + a} + \frac{1}{y_w^- + a}}}. \tag{5}$$

Among words with $|z_w| \geq 2.58$, each word's ten nearest polar neighbours generate an *attract* set A (same sign) and a *repel* set R (opposite sign). R also receives lexical antonyms found automatically in the vocabulary (w and `not_`w; w and un-, in-, dis-, im-, ir-w). Extending counter-fitting [8] and retrofitting [9], vectors are then adjusted by minimising

$$\mathscr{L}_{\mathrm{PA}} = \alpha \overline{\|\hat{\mathbf{v}}_w - \mathbf{v}_w^0\|^2} + \rho \overline{\left[\cos(\hat{\mathbf{v}}_a, \hat{\mathbf{v}}_b)\right]_+}\Big|_R + \kappa \overline{\left[\mu - \cos(\hat{\mathbf{v}}_a, \hat{\mathbf{v}}_b)\right]_+}\Big|_A, \tag{6}$$

where the first term preserves the original space, with $\alpha = 1$, $\rho = 1$, $\kappa = 0.3$ and $\mu = 0.6$. Because cosine lies in $[-1, 1]$ the repel term is zero exactly when a pair is orthogonal or opposed, so the loss stops pushing a pair once it is separated. The original method relies on external lexicons [8]; here constraints are derived from the corpus and its labels inside each fold.

4.3 Implementation and logic

The pipeline runs inside each training fold:

1. **Tokenize.** Lowercase, remove apostrophes and map numbers to `<num>`.
2. **Bind negation.** Join each negator to the next content word, skipping articles and intensifiers.
3. **Train SGNS.** Sample context pairs with a random window, sample negatives from the $3/4$-power noise distribution, and minimise Equation (3) with sparse gradients on the GPU.
4. **Score polarity.** Calculate z_w from training-fold labels only.
5. **Build constraints.** Create repel and attract pairs from polar neighbours and lexical antonyms, removing every evaluation pair.
6. **Counter-fit.** Minimise $\mathscr{L}_{\mathrm{PA}}$ with Adam, then renormalise the vectors.

Code 1: Negation binding (`src/telelens/text.py`)

```
def bind_negation(tokens):
    bound = []
    i = 0
    while i < len(tokens):
        token = tokens[i]
        if token in NEGATORS:
            j = i + 1
            while j < len(tokens) and tokens[j] in SKIP_AFTER_NEGATOR:
                j += 1
            if j < len(tokens) and tokens[j] not in STOPWORDS and tokens[j] not in NEGATORS:
                bound.append(f"not_{tokens[j]}")
                i = j + 1
                continue
        bound.append(token)
        i += 1
    return bound
```

Scanning left to right, a negator followed by a content word yields one token such as `not_helpful`. Function words after a negator, as in “not to”, are left unchanged, so only meaningful negations receive their own vectors.

Code 2: Skip-gram with negative sampling, written from scratch (`src/telelens/embeddings.py`)

```
def train_sgns(documents, dim=200, window=5, negatives=10, epochs=5, min_count=5, subsample=1e-3,
               lr=0.002, batch_size=8192, seed=42, device="cpu", bound=False):
    rng = np.random.default_rng(seed)
    torch.manual_seed(seed)
    counts = Counter(t for d in documents for t in d)
    words = [w for w, c in counts.most_common() if c >= min_count]
    index = {w: i for i, w in enumerate(words)}
    freq = np.array([counts[w] for w in words], dtype=np.float64)
    probs = freq / freq.sum()
    keep = np.minimum(1.0, np.sqrt(subsample / probs) + subsample / probs)
    noise = torch.tensor(freq**0.75 / (freq**0.75).sum(), dtype=torch.float, device=device)
    ids_docs = [np.array([index[t] for t in d if t in index], dtype=np.int64) for d in documents]

    centre = torch.nn.Embedding(len(words), dim, sparse=True, device=device)
    context = torch.nn.Embedding(len(words), dim, sparse=True, device=device)
    torch.nn.init.uniform_(centre.weight, -0.5 / dim, 0.5 / dim)
    torch.nn.init.zeros_(context.weight)
    optimizer = torch.optim.SparseAdam([*centre.parameters(), *context.parameters()], lr=lr)

    history = []
    for _ in range(epochs):
        c_np, o_np = _skipgram_pairs(ids_docs, window, keep, rng)
        order = rng.permutation(len(c_np))
        total, batches = 0.0, 0
        for start in range(0, len(order), batch_size):
            sel = order[start : start + batch_size]
            c = torch.as_tensor(c_np[sel], device=device)
            o = torch.as_tensor(o_np[sel], device=device)
            n = torch.multinomial(noise, len(sel) * negatives, replacement=True).view(len(sel),
     negatives)
            vc = centre(c)
            pos = (vc * context(o)).sum(1)
            neg = torch.bmm(context(n), vc.unsqueeze(2)).squeeze(2)
            loss = -(F.logsigmoid(pos) + F.logsigmoid(-neg).sum(1)).mean()
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total += loss.item()
            batches += 1
        history.append(total / max(batches, 1))
    return Embeddings(words, centre.weight.detach().cpu().numpy(), bound=bound), history
```

Two embedding tables hold target and context vectors. Frequent words are subsampled, and each batch computes the positive term and K negative terms of the SGNS loss in parallel using a batched matrix product, which is why training a fold takes seconds on a GPU.

Code 3: Polarity score from training labels (`src/telelens/embeddings.py`)

```
def log_odds_polarity(documents, labels, prior=0.01):
    pos, neg = Counter(), Counter()
    for doc, y in zip(documents, labels):
        (pos if y == 1 else neg).update(doc)
    vocab = set(pos) | set(neg)
    a0 = prior * len(vocab)
    n_pos, n_neg = sum(pos.values()), sum(neg.values())
    z = {}
    for w in vocab:
        yp, yn = pos[w], neg[w]
        delta = np.log((yp + prior) / (n_pos + a0 - yp - prior)) - np.log((yn + prior) / (n_neg + a0 -
     yn - prior))
        z[w] = float(delta / np.sqrt(1 / (yp + prior) + 1 / (yn + prior)))
    return z
```

Code 4: Polarity counter-fitting (`src/telelens/embeddings.py`)

```
def polarity_counterfit(emb, polarity, excluded_pairs=frozenset(), z_threshold=2.58, k=10,
                        repel_margin=0.0, attract_margin=0.6, repel_weight=1.0, attract_weight=0.3,
                        anchor_weight=1.0, steps=300, lr=0.01, device="cpu"):
    antonyms = lexical_antonyms(emb.words, excluded_pairs)
    attract, repel = polar_neighbour_constraints(emb, polarity, z_threshold, k)
    held_out = {tuple(sorted(p)) for p in excluded_pairs}
    attract = [p for p in attract if p not in held_out]
    repel = sorted({p for p in repel if p not in held_out} | set(antonyms))
    original = torch.tensor(emb.vectors, dtype=torch.float, device=device)
    vectors = original.clone().requires_grad_(True)
    optimizer = torch.optim.Adam([vectors], lr=lr)

    def ids(pairs):
        if not pairs:
            return None, None
        arr = torch.tensor([(emb.index[a], emb.index[b]) for a, b in pairs], device=device)
        return arr[:, 0], arr[:, 1]

    r_a, r_b = ids(repel)
    s_a, s_b = ids(attract)
    for _ in range(steps):
        normed = F.normalize(vectors, dim=1)
        loss = anchor_weight * (normed - original).pow(2).sum(1).mean()
        if r_a is not None:
            loss = loss + repel_weight * F.relu((normed[r_a] * normed[r_b]).sum(1) - repel_margin).mean
    ()
        if s_a is not None:
            loss = loss + attract_weight * F.relu(attract_margin - (normed[s_a] * normed[s_b]).sum(1)).
    mean()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    result = Embeddings(emb.words, F.normalize(vectors, dim=1).detach().cpu().numpy(), bound=emb.bound)
    stats = {"antonym_pairs": len(antonyms), "repel_pairs": len(repel), "attract_pairs": len(attract)}
    return result, stats
```

The loss has three parts: an anchor keeps vectors near the originals, a hinge pushes repel pairs until their cosine reaches zero, and a second hinge pulls attract pairs up to the margin. Removing `held_out` pairs before optimisation is what makes the evaluation honest.

Gensim Word2Vec and FastText with matching settings provide library baselines. Counter-fitting weights were selected on an inner validation split (Table 5); validation macro-F1 varied little, so this choice has limited influence.

Table 5: Counter-fitting grid on the inner validation split of fold 0. The top row became the default.

Repel	Attract	Anchor	Val. macro-F1	Ant. cos	Syn. cos
1.0	**0.3**	**1.0**	**0.8880**	**0.489**	**0.743**
2.0	0.3	1.0	0.8868	0.488	0.738
2.0	0.6	2.0	0.8867	0.488	0.743
0.5	0.3	4.0	0.8861	0.537	0.786
1.0	0.1	4.0	0.8848	0.501	0.747
1.0	0.3	2.0	0.8836	0.491	0.745

4.4 Results

Table 6 uses 16 synonym, 16 antonym and 10 negation pairs that were excluded from the constraints. Purity is the share of a word's ten nearest polar neighbours with the same polarity, with polarity calculated from the *held-out* fold, so the labels used for fitting cannot inflate it.

Table 6: Word2Vec evaluation across five grouped folds (mean ± s.d.). Evaluation word pairs were never used as constraints.

Model	Vocab	Syn. cos	Ant. cos	Neg. cos	Syn/Ant AUC	Purity	SimLex ρ
SGNS (scratch)	3758	0.833 ± 0.007	0.695 ± 0.011	–	0.817 ± 0.039	0.774 ± 0.003	0.107 ± 0.020
gensim Word2Vec	3758	0.728 ± 0.003	0.646 ± 0.006	–	0.696 ± 0.027	0.703 ± 0.006	0.069 ± 0.022
FastText	3758	0.728 ± 0.005	0.731 ± 0.003	–	0.542 ± 0.024	0.706 ± 0.011	0.140 ± 0.012
SGNS + negation	4050	0.828 ± 0.008	0.683 ± 0.015	0.688 ± 0.030	0.823 ± 0.046	0.773 ± 0.012	0.099 ± 0.012
PA-W2V	**4050**	**0.742 ± 0.015**	**0.373 ± 0.017**	**0.559 ± 0.061**	**0.863 ± 0.020**	**0.839 ± 0.021**	**0.096 ± 0.019**

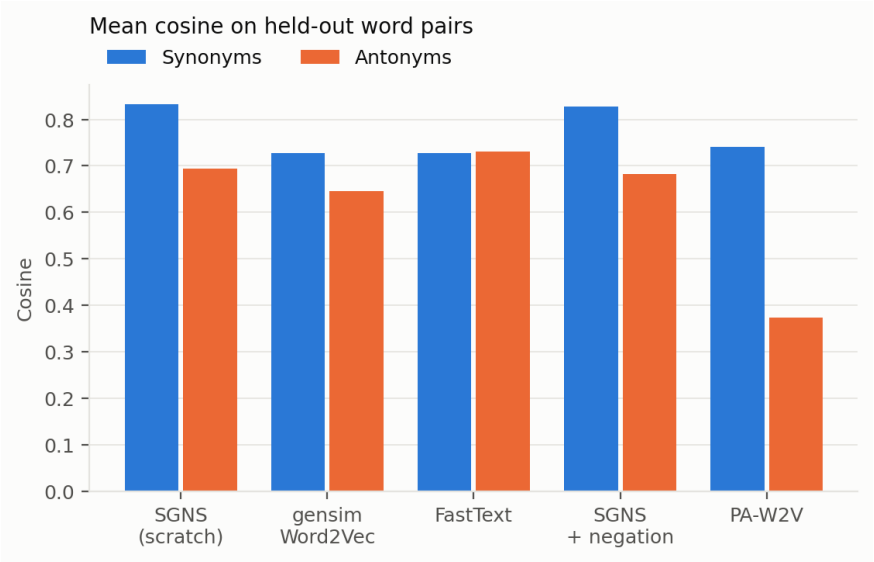


Figure 2: PA-W2V separates held-out antonyms from synonyms. FastText places antonyms as close as synonyms.

- **Opposites separate.** Mean antonym cosine falls from 0.683 to 0.373, and word–negation cosine from 0.688 to 0.559. The synonym-versus-antonym AUC rises from 0.823 to 0.863, while neighbour purity rises from 0.773 to 0.839.
- **There is a cost.** Synonym cosine also falls, from 0.828 to 0.742, so the space is less tightly clustered overall.
- **The from-scratch model is valid.** It matches or exceeds gensim on these checks. Its loss decreases monotonically by epoch (`sgns_loss.json`).
- **No change on SimLex-999 [27].** Spearman ρ stays near 0.10 for all models. SimLex is general English, and the corpus is small and domain-specific, so this benchmark shows no gain.
- **Pooling dilutes the effect.** Table 8 shows sentence cosine remains high after mean pooling, as most words are shared. The earlier report's concern is reduced at the word level but not solved at sentence level.

Table 7: Neighbour retrieval on held-out pairs. Synonym Hit@10 and MRR should be high; antonym Hit@10 counts an opposite word intruding among the ten nearest neighbours, so lower is better.

Model	Synonym Hit@10	Synonym MRR	Antonym Hit@10	Negation Hit@10
SGNS (scratch)	0.640	0.407	0.100	–
gensim Word2Vec	0.760	0.494	0.225	–
FastText	0.707	0.172	0.325	–
SGNS + negation	0.587	0.373	0.087	0.149
PA-W2V	**0.560**	**0.380**	**0.013**	**0.104**

Table 7 shows the trade-off in ranking terms. PA-W2V almost removes antonyms from the ten nearest neighbours (0.087 to 0.013, and FastText's 0.325 is far worse), but it retrieves slightly fewer synonyms (0.587 to 0.560). Gensim Word2Vec has the highest synonym Hit@10 (about 0.76) while admitting more antonyms, so no model wins on every measure.

Table 8: Sentence-level cosine after mean pooling. Averaging dilutes the word-level gain.

Sentence pair	SGNS	FastText	PA-W2V
the service is reliable / the service is not reliable	0.976	0.972	0.955
the connection is stable / the connection is unstable	0.990	0.993	0.957
the engineer was helpful / the engineer was not helpful	0.971	0.966	0.946
my problem was fixed / my problem was not fixed	0.975	0.965	0.963
broadband is slow and keeps disconnecting / internet connection is sluggish and drops	0.962	0.942	0.928

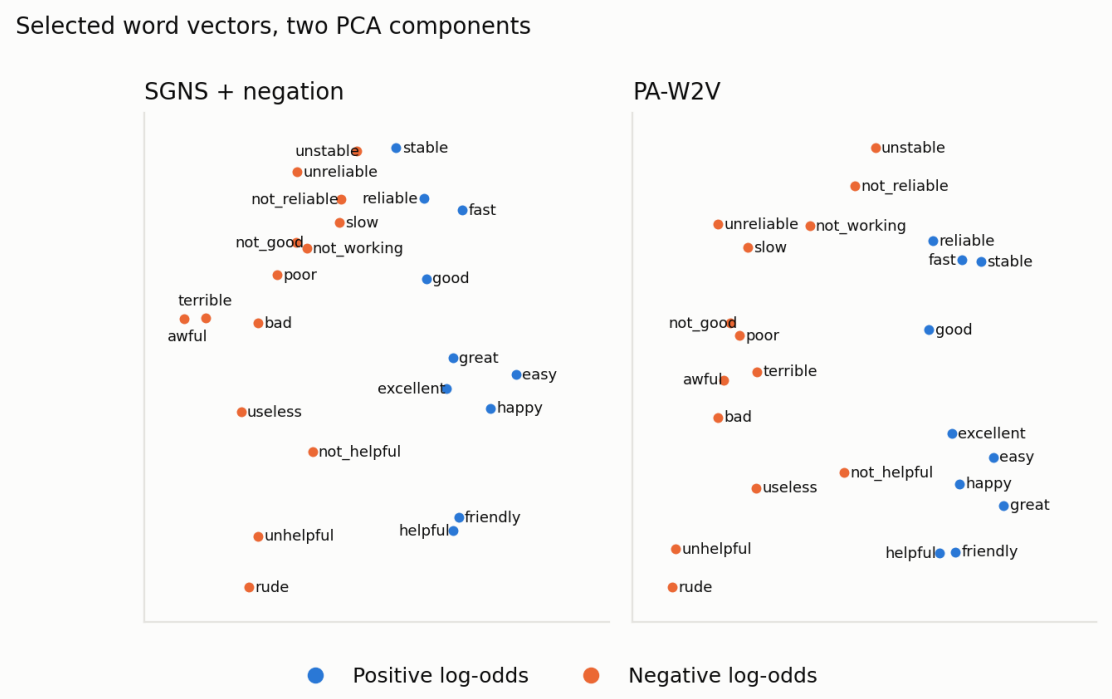


Figure 3: PCA of selected vectors. A two-dimensional projection discards information and is illustrative only; the quantitative evidence is in Table 6.

Table 9: Nearest neighbours on the whole-corpus model.

Word	Model	Top six neighbours
reliable	SGNS + negation	solid, stable, consistent, consistently, trustworthy, not_complaints
reliable	PA-W2V	consistently, trustworthy, lacking, rarely, importantly, mega
unreliable	SGNS + negation	disconnecting, reliability, dropouts, patchy, zoom, freezing
unreliable	PA-W2V	keeps, dropping, internet, disconnecting, awful, slow
stable	SGNS + negation	consistent, solid, rarely, reliable, slower, reliability
stable	PA-W2V	solid, competitors, pricey, rare, reliable, preferred
helpful	SGNS + negation	polite, patient, knowledgeable, professional, pleasant, courteous
helpful	PA-W2V	patient, polite, knowledgeable, courteous, knowledgable, kyle
bill	SGNS + negation	final, paper, payment, fees, declined, unpaid
bill	PA-W2V	final, paper, declined, fees, termination, advance

Table 9 also shows limits: *lacking* and *pricey* appear near positive words, reflecting co-occurrence rather

than meaning. Small-corpus embeddings remain noisy.

5 Case Study 2: Sentiment Classification

Abstract. Support teams need negative feedback found quickly and need to know when a prediction can be trusted. This case study compares nine classifiers on 9,874 telecom reviews under grouped five-fold validation, from TF–IDF and embedding features to fine-tuned DistilBERT, and adds calibrated selective triage with cross-fitted thresholds. Fine-tuned DistilBERT performs best, with macro-F1 0.916, ROC-AUC 0.974 and MCC 0.832, and is significantly better than TF–IDF (macro-F1 0.897). When reviews are ranked by predicted negativity, its top 100 are all truly negative and its top 500 are 99.8% negative. At a 5% risk target on unseen folds, the system achieved 4.93% error while handling 90.9% of reviews automatically. PA-W2V significantly improves embedding-only models, but adding it to TF–IDF gave no measurable gain, and idiomatic negation still defeats every model.

5.1 Methods

Reviews are labelled Bad or Good from star ratings. All models use the same grouped folds. Linear models use logistic regression with Platt calibration [14] fitted inside the training fold. DistilBERT [12] is fine-tuned for two epochs (learning rate 2×10^{-5}, 256 tokens) and calibrated by temperature scaling [13] on a 10% validation split; fitted temperatures of 1.09–1.29 confirm mild overconfidence before calibration. An untrained SST-2 DistilBERT measures what domain training adds.

5.2 Implementation and logic

1. **Split.** Create grouped folds so identical texts never cross the train and test boundary.
2. **Fit inside the fold.** Fit vectorizers, embeddings, polarity scores and classifiers on the training part only.
3. **Calibrate.** Use Platt scaling for linear models and temperature scaling for DistilBERT.
4. **Save.** Store every review's out-of-fold probability, so all metrics can be recalculated without retraining.
5. **Decide.** Choose a confidence threshold from the other folds, then automate or defer each review.

Code 5: Temperature scaling (`src/telelens/sentiment.py`)

```
def fit_temperature(logits, labels, device):
    logits = torch.tensor(logits, dtype=torch.float, device=device)
    labels = torch.tensor(labels, dtype=torch.long, device=device)
    log_t = torch.zeros(1, device=device, requires_grad=True)
    optimizer = torch.optim.LBFGS([log_t], lr=0.1, max_iter=100)

    def closure():
        optimizer.zero_grad()
        loss = torch.nn.functional.cross_entropy(logits / log_t.exp(), labels)
        loss.backward()
        return loss

    optimizer.step(closure)
    return float(log_t.exp().item())
```

Temperature scaling divides the logits by one learned value T, chosen to minimise cross-entropy on validation data. This changes confidence but never changes which class is predicted.

Code 6: Choosing the threshold for a target risk (`src/telelens/metrics.py`)

```
def threshold_for_risk(y_true, prob, target_risk):
    rows = risk_coverage(y_true, prob)
    valid = [r for r in rows if r["risk"] <= target_risk]
    return max(valid, key=lambda r: r["coverage"])["tau"] if valid else 1.01
```

Code 7: Cross-fitted deferral (`notebooks/02_sentiment_classification.ipynb`)

```
kept = np.zeros(len(y), dtype=bool)
for k in range(5):
    tau = threshold_for_risk(y[fold_of != k], p[fold_of != k], target)
    kept[fold_of == k] = np.maximum(p[fold_of == k], 1 - p[fold_of == k]) >= tau
```

The threshold for fold k is learned only from the other folds, so the achieved risk in Table 15 is measured on reviews that played no part in choosing it.

5.3 Results

Table 10: Out-of-fold sentiment results on 9,874 reviews (grouped five-fold).

Model	Accuracy	95% CI	Macro-F1	Bad recall	Bad prec.	ECE
DistilBERT fine-tuned	**0.916**	**0.911–0.921**	**0.916**	**0.910**	**0.922**	**0.024**
TF-IDF LR	0.897	0.891–0.903	0.897	0.920	0.881	0.013
TF-IDF + PA-W2V	0.896	0.890–0.902	0.895	0.922	0.877	0.019
TF-IDF + SGNS	0.891	0.885–0.897	0.891	0.918	0.872	0.024
PA-W2V mean LR	0.885	0.879–0.892	0.885	0.915	0.865	0.032
SGNS+neg mean LR	0.878	0.871–0.884	0.878	0.904	0.861	0.028
SGNS mean LR	0.871	0.865–0.878	0.871	0.902	0.852	0.031
FastText mean LR	0.857	0.850–0.864	0.857	0.895	0.834	0.036
DistilBERT SST-2 (no training)	0.823	0.815–0.831	0.819	0.971	0.751	0.159

Table 11: Threshold-free and agreement metrics from the saved out-of-fold probabilities.

Model	ROC-AUC	PR-AUC (Bad)	MCC	Kappa	Bal. acc.	Brier	Log loss
DistilBERT fine-tuned	**0.974**	**0.973**	**0.832**	**0.832**	**0.916**	**0.062**	**0.213**
TF-IDF LR	0.963	0.961	0.795	0.794	0.897	0.074	0.248
TF-IDF + PA-W2V	0.961	0.958	0.792	0.791	0.895	0.076	0.259
TF-IDF + SGNS	0.959	0.955	0.782	0.781	0.891	0.079	0.266
PA-W2V mean LR	0.947	0.939	0.772	0.771	0.885	0.089	0.299
SGNS+neg mean LR	0.946	0.939	0.757	0.756	0.878	0.091	0.304
SGNS mean LR	0.941	0.929	0.744	0.743	0.871	0.095	0.319
FastText mean LR	0.934	0.923	0.717	0.715	0.857	0.102	0.340
DistilBERT SST-2 (no training)	0.924	0.910	0.676	0.645	0.822	0.166	0.887

Table 12: Out-of-fold confusion counts (4,972 Bad and 4,902 Good reviews).

Model	Bad as Bad	Bad as Good	Good as Bad	Good as Good
DistilBERT fine-tuned	**4,524**	**448**	**383**	**4,519**
TF-IDF LR	4,572	400	618	4,284
TF-IDF + PA-W2V	4,583	389	642	4,260
TF-IDF + SGNS	4,562	410	669	4,233
PA-W2V mean LR	4,548	424	707	4,195
SGNS+neg mean LR	4,493	479	727	4,175
SGNS mean LR	4,484	488	781	4,121
FastText mean LR	4,448	524	884	4,018
DistilBERT SST-2 (no training)	4,830	142	1,605	3,297

Table 13: Triage queue hit rate: reviews are ranked by predicted negativity and the share of the top k that are truly Bad is reported. A queue of 2,000 can hold at most 40.2% of the 4,972 Bad reviews, so Bad recall@2000 cannot exceed 0.402.

Model	Hit@100	Hit@500	Hit@1000	Hit@2000	Recall@2000	First Good
DistilBERT fine-tuned	**1.000**	**0.998**	**0.995**	**0.993**	**0.399**	**470**
TF-IDF LR	1.000	0.994	0.994	0.988	0.397	294
TF-IDF + PA-W2V	0.990	0.992	0.991	0.985	0.396	94
TF-IDF + SGNS	0.980	0.986	0.989	0.982	0.395	42
PA-W2V mean LR	0.970	0.986	0.972	0.965	0.388	50
SGNS+neg mean LR	0.960	0.976	0.977	0.970	0.390	21
SGNS mean LR	0.980	0.962	0.967	0.962	0.387	10
FastText mean LR	0.950	0.960	0.968	0.961	0.386	4
DistilBERT SST-2 (no training)	1.000	0.970	0.958	0.938	0.377	115

Table 13 is the most direct measure of triage value. Ranking all reviews by predicted negativity, the top 100 are all truly Bad for DistilBERT, TF–IDF and the untrained SST-2 model, a **perfect Hit@100**. This perfect score applies only to the most confident 1% of reviews: DistilBERT's hit rate falls to 0.998 at 500 and 0.993 at 2,000, where the queue contains 39.9% of all Bad reviews, close to the 40.2% that a queue of that size can hold.

Hit@100 alone is an easy test, because even the untrained SST-2 model reaches 1.000, so it does not show that training helped. Deeper in the queue the models separate: the first Good review appears at rank 470 for DistilBERT, 294 for TF–IDF and 115 for SST-2, and their Hit@2000 values are 0.993, 0.988 and 0.938. The ROC-AUC of 0.974 confirms that strong ranking extends beyond the top of the list, while the SST-2 model's Brier score of 0.166 shows that a good ranking can still have poor probabilities. One caution applies: for the trained models, this ranking pools predictions from five separately fitted fold models. Calibration makes their scores comparable, but not perfectly so.

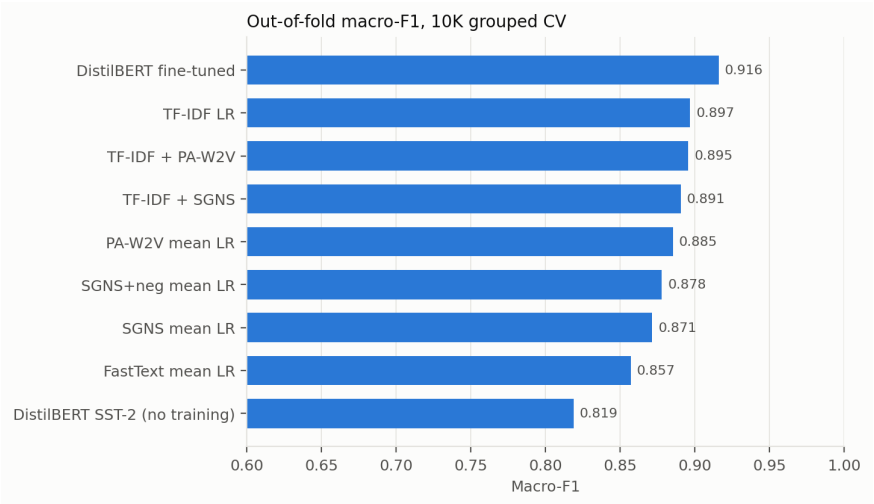


Figure 4: Out-of-fold macro-F1. The axis starts at 0.6 to show differences between models.

Table 14: Paired comparisons: bootstrap macro-F1 difference and exact McNemar test.

Comparison (A vs B)	F1 diff	95% CI	McNemar p
TF-IDF + PA-W2V vs TF-IDF LR	-0.0012	-0.0057 to +0.0032	0.5984
TF-IDF + PA-W2V vs TF-IDF + SGNS	+0.0049	+0.0011 to +0.0088	0.0184
PA-W2V mean LR vs SGNS mean LR	+0.0140	+0.0087 to +0.0193	<0.0001
PA-W2V mean LR vs SGNS+neg mean LR	+0.0076	+0.0029 to +0.0119	0.0007
SGNS+neg mean LR vs SGNS mean LR	+0.0064	+0.0022 to +0.0107	0.0062
DistilBERT fine-tuned vs TF-IDF LR	+0.0190	+0.0132 to +0.0249	<0.0001
DistilBERT fine-tuned vs TF-IDF + PA-W2V	+0.0202	+0.0144 to +0.0260	<0.0001

- **DistilBERT is the best classifier**: macro-F1 0.916, a difference of +0.019 over TF–IDF with a confidence interval excluding zero. Its confusion matrix is 4,524 / 448 for actual Bad and 383 / 4,519 for actual Good. It cost about 25 minutes of GPU time for five folds, against about 22 seconds for TF–IDF.
- **Each PA-W2V step improves embedding features.** SGNS 0.871 $\rightarrow$ with negation binding 0.878 $\rightarrow$ with counter-fitting 0.885, all significant. This is a real downstream gain for the Word2Vec contribution.
- **The TF–IDF hybrid adds nothing.** TF–IDF + PA-W2V (0.8955) and TF–IDF (0.8968) are statistically indistinguishable ($p = 0.60$). With bigrams available, averaged embeddings bring little extra.
- **Domain training matters.** The untrained SST-2 model reaches 0.819, with poor calibration (ECE 0.159).
- **Continuity.** On the earlier 1K file this project's TF–IDF model reaches macro-F1 0.882, compared with 0.8715 in the earlier report. Moving to 10K raises TF–IDF to 0.897.

5.4 Original contribution: selective triage

Rather than a label for every review, the system acts automatically only when confidence reaches a threshold τ, and defers others to an analyst. For selective classification [15], the risk at threshold τ is

$$R(\tau) = \frac{\sum_i \mathbf{1}[c_i \geq \tau]\,\mathbf{1}[\hat{y}_i \neq y_i]}{\sum_i \mathbf{1}[c_i \geq \tau]}, \qquad \tau^* = \min\{\tau : R(\tau) \leq r\}, \tag{7}$$

with $c_i = \max(p_i, 1 - p_i)$. To avoid reporting a threshold chosen on the data it is tested on, τ^* for each fold is selected using the other four. A support manager can then state an error tolerance r and receive the corresponding workload.

Table 15: Selective triage with cross-fitted thresholds: the requested risk is compared with the risk achieved on unseen folds.

Model	Target risk	Coverage	Achieved risk	Deferred	Bad recall (auto)
TF-IDF LR	0.10	0.989	0.099	112	0.923
TF-IDF LR	0.05	0.841	0.049	1568	0.966
TF-IDF LR	0.03	0.711	0.029	2849	0.980
TF-IDF LR	0.02	0.613	0.020	3818	0.985
TF-IDF + PA-W2V	0.10	0.987	0.099	125	0.926
TF-IDF + PA-W2V	0.05	0.821	0.051	1766	0.962
TF-IDF + PA-W2V	0.03	0.676	0.030	3200	0.977
TF-IDF + PA-W2V	0.02	0.534	0.019	4605	0.981
DistilBERT fine-tuned	0.10	1.000	0.084	0	0.910
DistilBERT fine-tuned	0.05	0.909	0.049	895	0.947
DistilBERT fine-tuned	0.03	0.817	0.032	1808	0.967
DistilBERT fine-tuned	0.02	0.717	0.019	2798	0.982

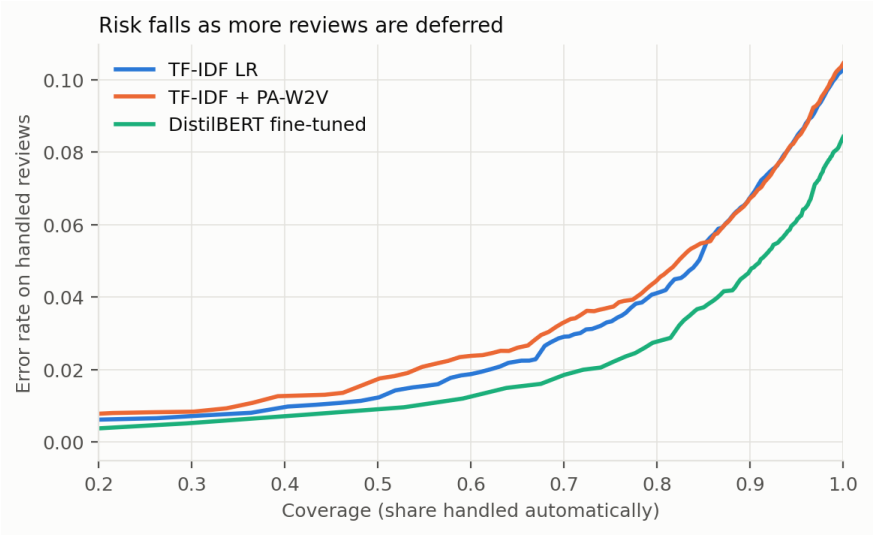


Figure 5: Risk against coverage. DistilBERT handles more reviews at every error level.

Achieved risk tracks the target closely on unseen folds. With DistilBERT at a 5% target, the system handles 90.9% of reviews with 4.93% error and sends 895 of 9,874 to an analyst. The earlier report's fixed-threshold triage has no such control.

5.5 Robustness

Table 16: Robustness: authored contrast set (30 sentences) and cross-domain test on Comcast complaints.

Model (trained on all 10K)	Contrast acc.	Comcast macro-F1	Comcast acc.	Comcast Bad recall
TF-IDF LR	0.667	0.685	0.985	0.994
TF-IDF + PA-W2V	0.733	0.675	0.984	0.993
PA-W2V mean LR	0.733	0.641	0.978	0.987
DistilBERT fine-tuned	0.867	0.699	0.982	0.988

On 30 authored minimal pairs, DistilBERT is correct on 86.7%, compared with 66.7% for TF–IDF. Embedding features help on simple negation ("the engineer was not helpful" receives $P(\text{Good})$ 0.93 under TF–IDF and 0.26 under the hybrid) but not on idioms: every model, DistilBERT included, still fails on "could not be better". This set is small and hand-written, so it is diagnostic only. On Comcast complaints Bad recall stays above 98%, but only 51 Good reviews exist, so macro-F1 of 0.64–0.70 rests on few examples. The dataset has no demographic attributes, so these tests are not a fairness assessment.

6 Case Study 3: Text Summarization

Abstract. Extractive summaries of customer reviews can silently distort what customers said. This case study introduces Polarity- and Aspect-Balanced MMR (PAB-MMR), which adds a sentiment-fidelity penalty and an aspect-coverage reward to Maximal Marginal Relevance, together with punctuation recovery for text that has none. On 47 telecom review clusters, compared with MMR, PAB-MMR reduced the polarity gap from 0.059 to 0.019, raised aspect recall from 0.680 to 0.850 and raised the sentiment hit rate from 0.702 to 0.936. Punctuation restoration raised sentence-boundary F1 from 0.299 to 0.828. Against human references on 20 FewSum products, ROUGE was statistically tied across all methods and PAB-MMR showed no sentiment advantage. The telecom gains are on metrics the method targets, so they show that it does what it was designed to do rather than that readers prefer its summaries.

6.1 Original contribution: PAB-MMR

Maximal Marginal Relevance [16] balances relevance and redundancy. Polarity- and Aspect-Balanced MMR adds two terms. Given source sentences with vectors $\mathbf{x}_i$, polarity p_i and aspect a_i, and a target share of positive sentiment π, it greedily selects under a word budget

$$s^* = \arg\max_{i \notin S} \Big[\lambda \cos(\mathbf{x}_i, \bar{\mathbf{x}}) - (1-\lambda) \max_{j \in S} \cos(\mathbf{x}_i, \mathbf{x}_j) + \beta\, w(a_i) \mathbf{1}[a_i \notin a(S)] - \eta\, \mathrm{JS}\big(\bar{p}_{S \cup i} \,\|\, \pi\big) \Big], \quad (8)$$

where $w(a)$ is the aspect's share of the cluster and $\bar{p}$ is the word-weighted mean polarity. Jensen–Shannon divergence with base-2 logarithms is bounded in $[0,1]$, is zero exactly when the distributions match, and is symmetric, so it is a stable penalty for greedy selection. Setting $\beta = \eta = 0$ recovers standard MMR, so the method is a strict generalisation and each term can be tested by ablation.

Aspects (billing, speed, support, installation, contract) use five seed lists expanded with PA-W2V neighbours (`aspect_lexicon.json`), connecting this task to Case Study 1. Telecom polarity comes from TF–IDF models cross-fitted across folds, so a review's own label never affects its sentence scores, connecting it to Case Study 2.

6.2 Implementation and logic

1. **Segment.** Split punctuated text directly; otherwise restore sentence boundaries with the punctuation model.
2. **Vectorize.** Represent each sentence with TF–IDF across the cluster.
3. **Score sentiment and aspect.** Attach a polarity to each sentence, and an aspect from the expanded lexicon.
4. **Select greedily.** At each step add the sentence with the best combined score that still fits the budget.
5. **Stop.** Finish when no remaining sentence fits within the word budget.

Code 8: PAB-MMR greedy selection (`src/telelens/summarize.py`)

```
def pab_mmr(vectors, sentences, budget, polarity=None, aspects=None, target=None, lam=0.7, beta=0.0, eta
    =0.0):
    x = normalize(vectors)
    relevance = x @ normalize(x.mean(0, keepdims=True)).ravel()
    sims = x @ x.T
    lengths = np.array([word_count(s) for s in sentences], dtype=float)
    aspect_share = {}
    if aspects is not None:
        values, counts = np.unique(aspects, return_counts=True)
        aspect_share = {v: c / counts.sum() for v, c in zip(values, counts)}
    if polarity is not None and target is None:
        target = float(np.average(polarity, weights=lengths))
    chosen, words, seen = [], 0, set()
    candidates = set(range(len(sentences)))
    while candidates:
        best, best_score = None, -np.inf
        for i in sorted(candidates):
            if words + lengths[i] > budget:
                continue
            redundancy_term = sims[i, chosen].max() if chosen else 0.0
            score = lam * relevance[i] - (1 - lam) * redundancy_term
            if aspects is not None and aspects[i] not in seen:
                score += beta * aspect_share.get(aspects[i], 0.0)
            if polarity is not None and eta:
                ids = chosen + [i]
                score -= eta * js_binary(float(np.average(polarity[ids], weights=lengths[ids])), target)
            if score > best_score:
                best, best_score = i, score
```

```
        if best is None:
            break
        chosen.append(best)
        words += lengths[best]
        candidates.remove(best)
        if aspects is not None:
            seen.add(aspects[best])
    return chosen
```

For each candidate, the score starts with standard MMR. A candidate from an unseen aspect gains a reward in proportion to that aspect's share of the cluster, and the penalty is calculated on the summary *as it would be if this sentence were added*, so the selection steers towards the cluster's true sentiment share as it goes.

Code 9: Sentence recovery with the punctuation model (`src/telelens/punctuate.py`)

```
def segment_many(self, texts):
    chunks, owners = [], []
    for owner, text in enumerate(texts):
        words = text.split()
        for start in range(0, len(words), self.window):
            chunks.append(words[start : start + self.window])
            owners.append(owner)
    order = sorted(range(len(chunks)), key=lambda i: len(chunks[i]))
    ordered = self._boundaries([chunks[i] for i in order])
    flags = {chunk: ordered[position] for position, chunk in enumerate(order)}
    output = [[] for _ in texts]
    current = [[] for _ in texts]
    for chunk_index in range(len(chunks)):
        owner = owners[chunk_index]
        for word, boundary in zip(chunks[chunk_index], flags[chunk_index]):
            current[owner].append(word)
            if boundary:
                output[owner].append(" ".join(current[owner]))
                current[owner] = []
    for owner, rest in enumerate(current):
        if rest:
            output[owner].append(" ".join(rest))
    return output
```

Long reviews are cut into 200-word windows and sorted by length for efficient batching, and a new sentence starts wherever the model predicts a full stop or question mark.

Code 10: Aspect lexicon expansion with PA-W2V neighbours (`src/telelens/summarize.py`)

```
def expand_aspects(embeddings, top_k=6, min_cosine=0.45):
    lexicon = {}
    for aspect, seeds in ASPECT_SEEDS.items():
        words = set(seeds)
        for seed in seeds:
            for word, score in (embeddings.neighbours(seed, top_k) if seed in embeddings else []):
                if score >= min_cosine and not word.startswith("not_") and len(word) > 2:
                    words.add(word)
        lexicon[aspect] = words
    return lexicon
```

6.3 Recovering sentences from unpunctuated text

The telecom corpus has no punctuation, so sentences must be recovered. To measure this with real ground truth, punctuation was stripped from 453 multi-sentence FewSum reviews and boundaries were compared with the originals.

Table 17: Sentence-boundary recovery after stripping punctuation from 453 FewSum reviews (tolerance two words).

Segmenter	Precision	Recall	F1	ROUGE-2 (test)
Cue words	0.254	0.179	0.194	0.047
Fixed 15-word windows	0.320	0.306	0.299	0.055
Punctuation restoration	**0.818**	**0.887**	**0.828**	**0.054**

The first design, a cue-word splitter, was *worse* than fixed windows and produced fragments. It was replaced with a pretrained punctuation model [20], which raises boundary F1 from 0.299 to 0.828. Fixed windows reach a similar ROUGE-2 on stripped text, as ROUGE rewards word overlap rather than readable sentences, so ROUGE alone would have missed this improvement.

6.4 Evaluation against human references

Using the FewSum gold summaries [26] and ROUGE [25], hyperparameters were selected on 40 products by ROUGE-2 and tested on 20. TextRank [24] and centroid selection serve as graph and frequency baselines. The word budget of 50 equals the mean reference length.

Table 18: FewSum test (20 products, three human references each): ROUGE F1.

Method	Words	ROUGE-1	ROUGE-2	95% CI (R-2)	ROUGE-L
Lead	48.7	0.267	0.041	0.029–0.055	0.155
Centroid	48.5	0.278	0.047	0.036–0.059	0.165
TextRank	48.3	0.281	0.045	0.036–0.055	0.164
MMR (tuned)	48.7	0.290	0.044	0.033–0.055	0.167
PAB-MMR (tuned)	48.5	0.277	0.045	0.035–0.056	0.163
PAB-MMR (fixed)	49.2	0.288	0.047	0.036–0.058	0.163
BART-large-CNN	41.9	0.265	0.043	0.029–0.061	0.163

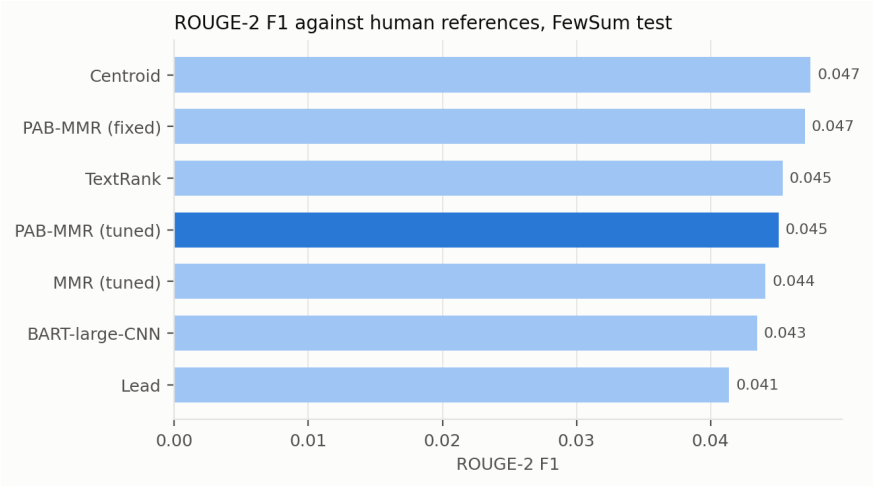


Figure 6: ROUGE-2 F1 on FewSum test. Confidence intervals overlap, so the methods are not separable.

All methods are statistically tied. PAB-MMR against MMR gives a mean ROUGE-2 difference of +0.001 ($p = 0.41$), and with 20 products this test has limited power. Scores are low across the board because human summaries paraphrase heavily and extractive methods cannot match unseen wording. The correct reading is that the balance terms *do not harm* reference similarity; they do not improve it.

Table 19: FewSum test: ROUGE precision and recall, word-distribution distance from the source, and sentiment agreement with the star ratings.

Method	R-1 P	R-1 R	R-2 P	R-2 R	Unigram JS	Sentiment hit
Lead	0.270	0.268	0.042	0.041	0.495	0.800
Centroid	0.283	0.276	0.048	0.047	0.476	0.750
TextRank	0.285	0.280	0.046	0.046	0.473	0.750
MMR (tuned)	0.296	0.289	0.045	0.044	0.463	0.650
PAB-MMR (tuned)	0.283	0.274	0.046	0.045	0.479	0.600
PAB-MMR (fixed)	0.289	0.290	0.047	0.047	0.464	0.750
BART-large-CNN	0.291	0.249	0.047	0.041	0.537	0.700

Table 19 also tests sentiment fidelity outside telecom, scoring whole summaries with the untrained SST-2 model against mean star ratings. Here PAB-MMR shows no advantage: the tuned version has the lowest sentiment hit (0.60) and the fixed version 0.75, against 0.80 for Lead. With 20 products one summary changes the rate by 0.05, and this scoring differs from the sentence scores PAB-MMR optimises, so the result should be read as no evidence of a benefit on this benchmark.

6.5 Evaluation on telecom digests

No human references exist for telecom clusters, so intrinsic measures are used over 47 clusters (up to 60 reviews each) with PAB-MMR weights fixed in advance ($\lambda = 0.7, \beta = 0.3, \eta = 0.5$). *Polarity gap* is the Jensen–Shannon divergence between summary sentiment and the true share of Good reviews; *aspect recall* is the share of the cluster's segments whose aspect appears in the summary.

Table 20: Telecom digests over 47 provider and provider-by-aspect clusters (no human references).

Method	Polarity gap	Aspect recall	Redundancy	Centroid cos	Words
Lead	0.046	0.653	0.030	0.280	48.1
Centroid	0.074	0.521	0.317	0.517	48.4
TextRank	0.065	0.562	0.290	0.518	48.5
MMR	0.059	0.680	0.081	0.576	48.3
PAB-MMR	**0.019**	**0.850**	**0.072**	**0.545**	**48.6**
BART-large-CNN	0.081	–	–	–	60.1

Table 21: Paired Wilcoxon tests across the 47 telecom clusters.

Metric	Versus	PAB-MMR	Other	p
Polarity gap	MMR	0.019	0.059	<0.0001
Aspect recall	MMR	0.850	0.680	<0.0001
Redundancy	MMR	0.072	0.081	0.1334
Polarity gap	TextRank	0.019	0.065	<0.0001
Polarity gap	BART-large-CNN	0.019	0.081	0.0002

Table 22: Telecom clusters: sentiment hit (summary and cluster agree on the majority sentiment), unigram JS divergence from the source, and compression ratio.

Method	Sentiment hit	Unigram JS	Compression
Lead	0.681	0.615	0.0246
Centroid	0.681	0.588	0.0248
TextRank	0.702	0.582	0.0249
MMR	0.702	0.529	0.0248
PAB-MMR	**0.936**	**0.534**	**0.0249**
BART-large-CNN	0.809	0.557	0.0527

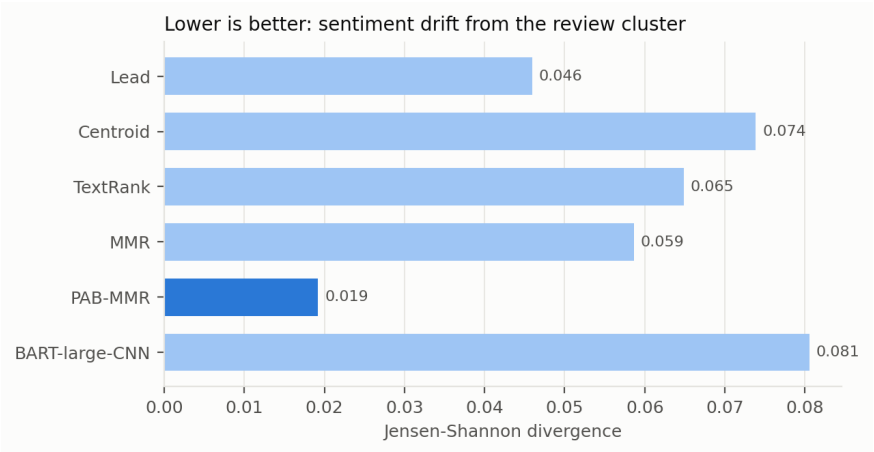


Figure 7: Sentiment drift from the source cluster. BART is evaluated with the sentence-polarity model trained on all reviews.

PAB-MMR reduces the polarity gap by 67% against MMR (0.059 to 0.019) and raises aspect recall from 0.680 to 0.850, both significant. Its digests agree with the cluster's majority sentiment in 93.6% of clusters, against 70.2% for MMR and 80.9% for BART. The cost is a small fall in centroid similarity (0.576 to 0.545), and word-distribution distance stays about the same (0.529 to 0.534). **An important caveat:** polarity gap and aspect recall are what PAB-MMR explicitly optimises, so this shows that the

method does what it is designed to do, not that human readers prefer it. A reader study would be needed for that claim.

Table 23: Generated digests (lowercase source text, segments separated by bars).

Cluster	Summary
virgin PAB-MMR	i had been a loyal virgin media customer for years | i have been with virgin broadband for a few months now and cant wait for my contract to end | i have found their internet service and customer service superb | when virgin media works it can be a very good service
virgin MMR	i had been a loyal virgin media customer for years | i have been with virgin broadband for a few months now and cant wait for my contract to end | at present the service is not intermittent | when virgin media works it can be a very good service
bt PAB-MMR	so another call to bt i made | i have been a customer of zen internet for ten years | later i was told they will get the original phone number back once the phone line is restored | really happy with the service and product received from bt shop
bt MMR	so another call to bt i made | later i was told they will get the original phone number back once the phone line is restored | i have been a customer of zen internet for ten years | really happy with the service and product received from bt shop

BART-large-CNN [17], a news model, tends to copy from the start of its truncated input (novel token rate 2.4%) and in one cluster produced text about other providers, as the concatenated input mixes mentions. Extractive digests remain directly traceable to source reviews, which supports human verification. The digests also show remaining weaknesses: sentences can be off-topic, and provider clusters are found by keyword, so a review mentioning a competitor can enter the wrong cluster.

7 Case Study 4: Factoid Question Answering

Abstract. A support analyst often needs one fact from thousands of complaints, and a system that always answers will invent one when it is absent. This case study builds a retriever–reader pipeline over 5,629 Comcast complaints, with answer-type gating and calibrated abstention, and evaluates it on 2,000 SQuAD questions and a new 132-question domain set. The RoBERTa reader reached exact match 0.844 on SQuAD. On 68 held-out answerable telecom questions, the end-to-end system reached exact match 0.618 and Hit@5 0.691, with document retrieval at Hit@5 0.824 and MRR 0.703. Abstention raised overall accuracy on all 88 held-out questions from 0.477 to 0.602 and rejected 80% of unanswerable questions, at an answerability AUROC of 0.847. When the correct complaint was retrieved, exact match reached 0.894, and 21 of 26 errors came from retrieving the wrong complaint. Word2Vec fusion weakened fact retrieval, a useful negative result. The domain set has one annotator, and its test portion was not strictly blind.

7.1 Pipeline

The system follows the classic retriever–reader architecture for factoid QA [28]:

1. **Question processing.** Rules map a question to an expected type: MONEY, DURATION, COUNT, TIME, DATE or ENTITY.
2. **Passage retrieval.** Complaints are split into three-sentence overlapping passages, indexed by BM25 [18], with dense and fused alternatives.
3. **Answer extraction.** A SQuAD-trained reader [19,22] scores spans in the top ten passages.
4. **Answer-type gate (original).** Spans that do not fit the expected type are discarded, so a MONEY question cannot return “Modem”.
5. **Calibrated abstention (original).** Each candidate receives

$$\text{score} = (s_{\text{start}} + s_{\text{end}} - s_{\text{CLS}}) + \frac{w}{\text{rank}}, \tag{9}$$

the reader's advantage over its no-answer option plus a retrieval bonus. If the best score falls below a threshold τ tuned on development questions, the system answers "not found in the corpus". Every answer is returned with its source complaint.

7.2 Implementation and logic

Code 11: Question processing: expected answer type (`src/telelens/qa.py`)

```
def answer_type(question):
    q = question.lower()
    if re.search(r"\bat what time\b|\bwhat time\b", q):
        return "TIME"
    if re.search(r"\bwhat date\b|\bwhich date\b|\bwhen did\b|\bearliest date\b", q):
        return "DATE"
    if re.search(r"how long|how far|how often|how many (?:seconds|minutes|hours|weeks|months|years|days)
     |minimum hold|wait time", q):
        return "DURATION"
    if re.search(r"how much|what (?:\w+ ){0,3}(?:price|fee|charge|bill|deposit|credit|refund|rate|amount
     |cost|rebate)\b|what did .* (?:cost|rise|increase)", q):
        return "MONEY"
    if re.search(r"how many|how old", q):
        return "COUNT"
    return "ENTITY"
```

Ordered rules map question wording to a type. Time and date are checked first, and duration before money, so "how long" and "how many days" are not mistaken for amounts.

Code 12: Passage retrieval: BM25, dense, fusion and reranking (`src/telelens/qa.py`)

```
def rank(self, question, mode="bm25", k=100):
    if mode == "bm25":
        return [int(i) for i in np.argsort(-self.bm25.scores(retrieval_tokens(question)))[:k]]
    if mode in self.spaces:
        return [int(i) for i in np.argsort(-self._dense(question, mode))[:k]]
    if mode.startswith("rerank:"):
        _, dense, gamma = mode.split(":")
        lexical = self.bm25.scores(retrieval_tokens(question))
        top = np.argsort(-lexical)[:100]
        semantic = self._dense(question, dense)[top]
        z = lambda v: (v - v.mean()) / (v.std() + 1e-9)
        order = np.argsort(-(z(lexical[top]) + float(gamma) * z(semantic)))
        return [int(i) for i in top[order][:k]]
    dense = mode.split("+")[1]
    a = np.argsort(-self.bm25.scores(retrieval_tokens(question)))[:500]
    b = np.argsort(-self._dense(question, dense))[:500]
    fused = Counter()
    for ranking in (a, b):
        for position, passage in enumerate(ranking):
            fused[int(passage)] += 1.0 / (self.rrf_k + position + 1)
    return [p for p, _ in fused.most_common(k)]
```

The method supports four strategies. Equal-weight fusion combines BM25 and dense rankings by reciprocal rank. Reranking is gentler: it keeps BM25's top 100 and adds a small weighted dense score, so embeddings can only reorder passages that BM25 already found.

Code 13: Reading, gating and scoring (`src/telelens/qa.py`)

```
def answer(self, question, gate=True, retrieval_weight=2.0):
    ranked, top, contexts, all_spans = self.read(question)
    kind = answer_type(question)
    best = {"answer": "", "score": -1e9, "doc_id": None, "evidence": "", "type": kind}
    for position, spans in enumerate(all_spans):
        for span in spans:
            if gate and not passes_gate(kind, span["text"]):
                continue
            score = span["margin"] + retrieval_weight / (position + 1)
            if score > best["score"]:
```

```
                best = {"answer": span["text"], "score": score, "doc_id": self.retriever.passages[top[
    position]]["doc_id"],
                        "evidence": contexts[position], "type": kind}
    return best, ranked
```

Each span keeps its advantage over the reader's no-answer option, plus a bonus for appearing in a highly ranked passage. Spans that fail the type gate are skipped, and the best remaining span keeps its source passage as evidence.

Code 14: Evaluating abstention (`src/telelens/metrics.py`)

```
def selective_qa(items, tau):
    answerable = [i for i in items if i["answerable"]]
    unanswerable = [i for i in items if not i["answerable"]]
    answered = [i for i in answerable if i["score"] >= tau]
    correct = sum(i["em"] for i in answered)
    false_answers = sum(i["score"] >= tau for i in unanswerable)
    return {
        "tau": float(tau),
        "overall_accuracy": (correct + len(unanswerable) - false_answers) / max(len(items), 1),
        "answer_coverage": len(answered) / max(len(answerable), 1),
        "answered_em": correct / max(len(answered), 1),
        "answered_f1": float(np.mean([i["f1"] for i in answered])) if answered else 0.0,
        "unanswerable_rejection": 1 - false_answers / max(len(unanswerable), 1),
    }
```

Code 15: Ranking metrics (`src/telelens/metrics.py`)

```
def retrieval_scores(ranks, ks=(1, 3, 5, 10, 20)):
    ranks = [r if r is not None else np.inf for r in ranks]
    scores = {f"hit@{k}": float(np.mean([r <= k for r in ranks])) for k in ks}
    scores["mrr"] = float(np.mean([0.0 if np.isinf(r) else 1.0 / r for r in ranks]))
    scores["ndcg@10"] = float(np.mean([1.0 / np.log2(r + 1) if r <= 10 else 0.0 for r in ranks]))
    return scores
```

7.3 Why a new gold set was needed

No public factoid set exists for telecom complaints. A set of 132 questions was therefore written against seeded samples of complaints (`qa_gold.jsonl`). Every answer was checked programmatically to occur verbatim in its source complaint. Thirty questions ask for facts the corpus should not contain. Questions were split into 44 development and 88 test before any system was run.

7.4 Public benchmark

Table 24: Reader benchmark on a seeded 2,000-question SQuAD v1.1 dev sample with the gold paragraph supplied.

Reader	EM	95% CI	F1	Seconds
DistilBERT-SQuAD	0.785	0.767–0.802	0.860	25
RoBERTa-SQuAD2	0.844	0.829–0.860	0.907	42

Both readers perform in a credible range on SQuAD [21], which confirms correct integration before domain evaluation. RoBERTa scores higher here, and development selection on telecom questions chose it as well.

7.5 Retrieval

Table 25: Retrieval of the evidence complaint among 5,629 documents (68 held-out answerable questions). With one relevant document per question, Hit@k equals Recall@k and MAP equals MRR.

Retriever	Hit@1	Hit@3	Hit@5	Hit@10	Hit@20	MRR	nDCG@10
BM25	0.603	0.765	0.824	0.853	0.868	0.699	0.735
SGNS dense	0.162	0.235	0.324	0.412	0.500	0.234	0.269
PA-W2V dense	0.132	0.191	0.265	0.353	0.397	0.192	0.225
BM25 + SGNS	0.250	0.544	0.618	0.779	0.853	0.427	0.505
BM25 + PA-W2V	0.235	0.471	0.559	0.750	0.853	0.391	0.468
BM25 rerank SGNS 0.25	0.603	0.779	0.824	0.853	0.868	0.703	0.738
BM25 rerank SGNS 0.5	0.603	0.765	0.824	0.853	0.868	0.700	0.735
BM25 rerank PA-W2V 0.25	0.603	0.779	0.838	0.853	0.882	0.697	0.733
BM25 rerank PA-W2V 0.5	0.588	0.779	0.838	0.853	0.882	0.695	0.732

This is a negative result for embeddings. Dense retrieval is far weaker than BM25, and equal-weight reciprocal rank fusion [23] damages BM25 (MRR 0.699 to about 0.40), because factoid questions depend on exact numbers and rare names, which static vectors encode poorly (all numbers become one token). Gentle reranking of BM25's top 100 neither helps nor harms measurably. Word2Vec suits sentiment and aspect expansion here, but not precise fact retrieval. Questions written while reading a complaint also share its wording, which favours BM25 (Section 10).

7.6 End-to-end results

Configuration was selected on development questions: BM25 reranked with SGNS at weight 0.25, RoBERTa, gate on, $w = 2$, $\tau = 3.27$.

Table 26: Ablation on 68 held-out answerable questions.

System	EM	95% CI	F1
Rule extractor (regex types)	0.324	0.221–0.441	0.338
BM25 + RoBERTa-SQuAD2, no gate	0.588	0.471–0.706	0.632
BM25 + RoBERTa-SQuAD2, gate	0.618	0.500–0.735	0.642
BM25+SGNS + RoBERTa-SQuAD2, gate	0.544	0.426–0.662	0.561
BM25+PA-W2V + RoBERTa-SQuAD2, gate	0.500	0.382–0.618	0.532
BM25 rerank PA-W2V 0.25 + RoBERTa-SQuAD2, gate	0.632	0.515–0.735	0.650
Dev-selected retriever + DistilBERT-SQuAD, gate	0.574	0.456–0.677	0.596
Dev-selected system	**0.618**	**0.500–0.735**	**0.642**

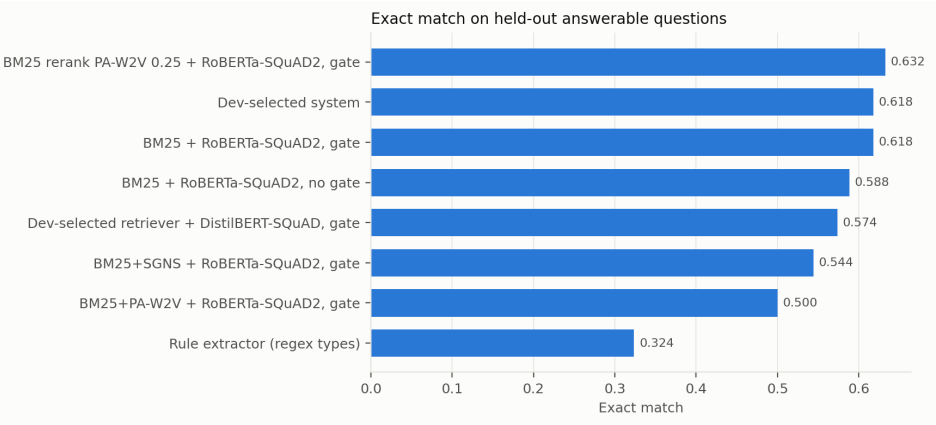


Figure 8: Exact match by system on held-out answerable questions.

A reader clearly beats regular-expression extraction (0.618 against 0.324, with non-overlapping intervals). The gate adds about three points (0.588 to 0.618), but with 68 questions the intervals are wide ($\pm$0.12), so that gain and the differences among reader systems are suggestive only.

Table 27: Abstention on all 88 held-out questions (68 answerable, 20 unanswerable).

Policy	Overall acc.	Coverage	Answered EM	Answered F1	Rejected
Always answer	0.477	1.000	0.618	0.642	0.000
Abstain below $\tau = 3.27$	**0.602**	**0.721**	**0.755**	**0.769**	**0.800**

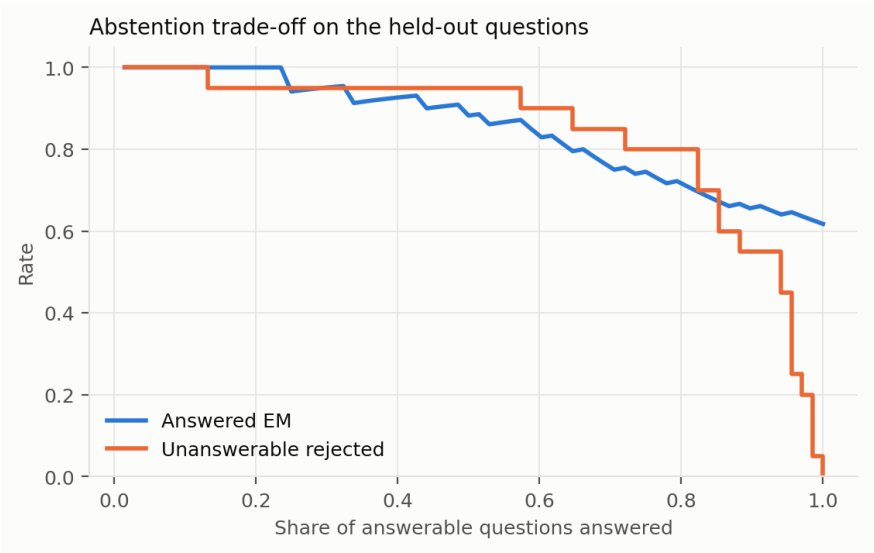


Figure 9: Lowering the threshold answers more questions but rejects fewer unanswerable ones.

Without abstention, the system invents an answer for every unanswerable question. With the threshold chosen on development data, it rejects 80% of them, and answered exact match rises from 0.618 to 0.755, while still answering 72% of answerable questions. Overall accuracy rises from 0.477 to 0.602. This is the largest practical gain in the QA case study.

Table 28: Answer Hit@k: the correct answer appears among the top k ranked spans.

Match rule	Hit@1	Hit@3	Hit@5	Hit@10
Exact match	0.618	0.676	0.691	0.706
Token F1 $\geq$ 0.5	0.662	0.706	0.750	0.750

Table 29: Additional held-out metrics for the dev-selected system.

Metric	Value
Answerability AUROC (score separates answerable from unanswerable)	0.847
Answerability average precision	0.941
Rejection precision / recall / F1 at τ	0.457 / 0.800 / 0.582
Overall accuracy 95% CI with abstention	0.500–0.705
Answer drawn from the correct complaint	0.691
Exact match when the correct complaint is used	0.894
Errors from the wrong complaint / from the right complaint	21 / 5
Gate vs no gate: correct only with / only without; McNemar p	2 / 0; 0.500

Table 30: Dev-selected system by detected answer type.

Type	Questions	EM	F1
COUNT	1	1.000	1.000
DATE	2	0.500	0.500
DURATION	13	0.615	0.692
ENTITY	24	0.625	0.625
MONEY	26	0.577	0.603
TIME	2	1.000	1.000

The type table is split thinly: COUNT and TIME score 1.000 on only one and two questions, so these are not meaningful perfect scores. Table 29 shows that the abstention score separates answerable from unanswerable questions well (AUROC 0.847). Rejection precision is only 0.457, though, because abstention also turns away many answerable questions: 19 answerable questions are rejected alongside the 16 unanswerable ones. The gate changed the result on only two questions, both in its favour, which is too few to be significant ($p = 0.50$).

7.7 Error analysis

Table 29 locates the main weakness. Answers drawn from the correct complaint are right 89.4% of the time, while answers from a wrong complaint are never exactly right, and 21 of the 26 errors come from a wrong complaint. Retrieval, not reading, is therefore where future work should concentrate. Inspection of errors (`test_errors.csv`) shows three main causes:

- **Wrong complaint retrieved.** Many complaints share the same pattern, so "How much in lost wages…" can find a similar case with another figure. A plausible amount from the wrong complaint fits the expected type and passes the gate.
- **Span boundaries.** "158.00 a month" against gold "158.00" is correct in substance but counted wrong by exact match; token F1 captures this partly.
- **Reader confusion on complaint language.** "Which provider…" can return Comcast itself, since the reader lacks the idea of "another provider".

8 Integrated System

All four components run together in `05_integrated_system.ipynb`, which also runs the unit tests (`tests/test_core.py`). Outputs are recorded in `demo.json`.

Code 16: Integrated triage operation (`src/telelens/pipeline.py`)

```
def triage(self, review, k=3):
    prob = float(self.good_probability([review])[0])
    confidence = max(prob, 1 - prob)
    vector = normalize(self.vectorizer.transform([review]))
    scores = (self.history_vectors @ vector.T).toarray().ravel()
    order = np.argsort(-scores)[:k]
    return {
        "prediction": "Good" if prob >= 0.5 else "Bad",
        "probability_good": prob,
        "decision": "automatic" if confidence >= self.defer_threshold else "defer_to_analyst",
        "similar": [{"cosine": float(scores[i]), "label": "Good" if self.history_labels[i] else "Bad",
                     "text": self.history_texts[i][:160]} for i in order],
    }
```

Table 31: Integrated system: live triage output.

Incoming review	Prediction	P(Good)	Decision
my broadband keeps disconnecting and customer service has not fixed it	Bad	0.039	automatic
the engineer arrived on time and the router was set up in minutes	Good	0.939	automatic
it was fine i suppose though the price went up after a few months	Good	0.861	automatic
the speed is good but nobody answers when you call and the bill was wrong once	Good	0.800	automatic
great deal at first however the router needed replacing twice and they sorted it eventually	Good	0.982	automatic

Table 32: Integrated system: live question answering with evidence or abstention.

Question	Answer	Score	Complaint
What was the cancellation fee for the customer who moved to a neighborhood not serviced by Comcast?	$756.82	11.86	2562
Which insurance company refused to pay the full electrician estimate after faulty cable wiring?	Liberty Mutual	10.58	4307
How much does Comcast charge for 5G home internet in Seattle?	(abstained)	2.91	–

Digest for Virgin reviews: *im not happy with virgin media | i had phone call to virgin mobile customer services just to change my but i had silly conversation and no have been virgin broadband around more than years and last year i decided to change my mobile to virgin and now i am realy p off with your | i want my contract cancelled*

The integrated system uses TF–IDF for live triage because it is fast and explainable; DistilBERT remains the most accurate option where a GPU is available. The demonstration exposes a limitation: the mixed review "the speed is good but nobody answers when you call" is confidently labelled Good and handled automatically, so deferral did not catch it. Deferral lowers error in aggregate (Table 15) but does not guarantee catching any individual review.

9 Comparison with Supplied References

Reported scores from different datasets cannot be ranked directly because labels, sample sizes and protocols differ.

Table 33: Critical comparison with the four supplied papers.

Reference	Focus and reported contribution	Relationship to this project
[1] Abdelmotaleb et al. (2026)	Compares TF–IDF, Word2Vec, FastText, Doc2Vec, BERT, classifiers and PCA pooling for telecom star ratings, with computational costs.	Source of the dataset and of the accuracy-versus-cost framing. This project uses the 10K file released with it, not the 367,000-review corpus. It adds selective deferral, summarization and QA.
[2] Siswa et al. (2026)	Compares TF–IDF, Word2Vec and FastText with classical classifiers on 3,441 Indonesian comments; reports 85.2% for SVM + FastText.	Here FastText was the weakest embedding feature (0.857), which reinforces that representation choice depends on domain and language.
[3] Joseph and Odumuyiwa (2026)	Surveys text representations from classical vectors to sentence embeddings.	Provides the conceptual framing. The pooling result here (Table 8) illustrates its point about matching representation granularity to the task.
[4] Khare and Pandey (2026)	Reviews Word2Vec, Bi-LSTM and transformer department classification in healthcare, with overfitting concerns.	Motivates this project's emphasis on leakage control, cross-fitting and held-out evaluation.

10 Threats to Validity and Disclosed Limitations

- **Single-annotator QA gold.** One annotator wrote the 132 questions, reading the complaints directly, so they share vocabulary with their sources. This favours lexical retrieval, and the 88 test questions give wide confidence intervals.
- **Changes after seeing test results.** These are disclosed in full: (i) the answer-type pattern was broadened after a test error revealed a bug, leaving the dev-selected exact match unchanged at 0.618; (ii) reranking modes were added after a first run in which fusion performed badly, though selection remained on development data; (iii) the cue-word segmenter was replaced after its measured failure on the segmentation benchmark; (iv) after the first summarization run, BART's telecom length limit was lowered towards the word budget and the minimum telecom segment length was raised from four to five words. The QA test set is therefore not strictly blind. The Hit@k, nDCG, AUROC, MCC and related metrics were also added after the first results were seen. They are reported for every model, and no model, weight or threshold was changed because of them.
- **Self-aligned metrics.** PAB-MMR's telecom advantage is on metrics it optimises. No reader study has been conducted.
- **Small benchmarks.** FewSum test has 20 products; the contrast set has 30 sentences; Comcast has 51 Good reviews.
- **Embedding held-out pairs are curated.** They were chosen by hand from common telecom words. A different list would change the numbers.
- **Keyword-based clusters.** Provider and aspect assignment is lexical and has not been separately validated.
- **Labels are proxies.** Star ratings do not always reflect the text, and the data comes from one review source per corpus, so results may not generalise across providers or countries.
- **No operational evidence.** No claim is made about reduced waiting time or cost; a field evaluation would be required.

11 Conclusion and Future Work

This case study delivers all four assigned topics as one working, evaluated system. Its main findings are:

1. **Word2Vec.** Plain embeddings confuse opposites. PA-W2V cut held-out antonym similarity from 0.683 to 0.373 and improved embedding-based sentiment significantly, though sentence-level pooling hides much of the gain and SimLex shows no change.
2. **Sentiment.** Fine-tuned DistilBERT achieved macro-F1 0.916, ROC-AUC 0.974 and MCC 0.832, and its 100 most negative predictions were all correct. Cross-fitted selective triage met its 5% risk target on unseen folds while automating 91% of reviews.
3. **Summarization.** PAB-MMR reduced sentiment drift by 67% and increased aspect recall without measurable harm to ROUGE. Punctuation restoration made unpunctuated reviews usable, with boundary F1 of 0.83.
4. **Factoid QA.** Answer gating and abstention rejected 80% of unanswerable questions and raised overall accuracy from 0.477 to 0.602. BM25 outperformed embeddings for retrieval, which is a useful negative result.

The main lesson is that each method should be chosen by evidence for each task. Embeddings improved sentiment representation and failed at fact retrieval, and the most valuable feature across tasks was knowing when not to decide.

Next steps are to obtain independent annotation of the QA gold set with agreement statistics, run a reader study on digests, use contextual embeddings for sentence-level negation, replace keyword clusters with validated aspect models, and evaluate the triage policy with support analysts before making any operational claim.

References

[1] H. Abdelmotaleb, C. Mcneile, and M. Wojtyś, "A comparative study of word embedding techniques for classification of star ratings," *Expert Systems with Applications*, vol. 297, article 129037, 2026. doi: 10.1016/j.eswa.2025.129037.

[2] T. A. Y. Siswa, M. Fattah, M. A. Fahrozi, D. F. Rahman, and F. I. Ramadhani, "Optimizing Sentiment Classification on IKN Development: A Comparative Study of TF-IDF, Word2Vec, and FastText Embeddings," *Engineering Proceedings*, vol. 137, no. 20, 2026. doi: 10.3390/engproc2026137020.

[3] P. K. Joseph and V. Odumuyiwa, "From word to sentence embedding and beyond: Bridging the gap in text representation survey," *Natural Language Processing Journal*, vol. 15, article 100214, 2026. doi: 10.1016/j.nlp.2026.100214.

[4] H. Khare and D. Pandey, "Deep Neural Architectures for Automated Department Classification: A Review of Word2Vec, Bi-LSTM, and Transformer-Based Models in Healthcare," in *2026 IEEE 6th IC2PCT*, 2026. doi: 10.1109/IC2PCT68894.2026.11584372.

[5] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, 2013.

[6] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *NeurIPS*, 2013.

[7] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *TACL*, vol. 5, pp. 135–146, 2017.

[8] N. Mrkšić et al., "Counter-fitting word vectors to linguistic constraints," in *NAACL-HLT*, 2016.

[9] M. Faruqui et al., "Retrofitting word vectors to semantic lexicons," in *NAACL-HLT*, 2015.

[10] B. L. Monroe, M. P. Colaresi, and K. M. Quinn, "Fightin' words: Lexical feature selection and evaluation for identifying the content of political conflict," *Political Analysis*, vol. 16, no. 4, 2008.

[11] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *EMNLP*, 2002.

[12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT," arXiv:1910.01108, 2019.

[13] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *ICML*, 2017.

[14] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, 1999.

[15] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *NeurIPS*, 2017.

[16] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in *SIGIR*, 1998.

[17] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *ACL*, 2020.

[18] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.

[19] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[20] O. Guhr, A.-K. Schumann, F. Bahrmann, and H. J. Böhme, "FullStop: Multilingual punctuation prediction," in *SwissText*, 2021.

[21] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *EMNLP*, 2016.

[22] P. Rajpurkar, R. Jia, and P. Liang, "Know what you don't know: Unanswerable questions for SQuAD," in *ACL*, 2018.

[23] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *SIGIR*, 2009.

[24] R. Mihalcea and P. Tarau, "TextRank: Bringing order into text," in *EMNLP*, 2004.

[25] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *ACL Workshop on Text Summarization Branches Out*, 2004.

[26] A. Bražinskas, M. Lapata, and I. Titov, "Few-shot learning for opinion summarization," in *EMNLP*, 2020.

[27] F. Hill, R. Reichart, and A. Korhonen, "SimLex-999: Evaluating semantic models with (genuine) similarity estimation," *Computational Linguistics*, vol. 41, no. 4, 2015.

[28] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft.

A Evidence Index

Each headline claim maps to a file in the public repository.

Claim	Evidence file
Word2Vec metrics, five folds	results/word2vec/intrinsic_by_fold.csv
Counter-fitting grid	results/word2vec/counterfit_tuning.csv
Sentiment scores and intervals	results/sentiment/scores.csv
ROC-AUC, MCC, Brier and related metrics	results/sentiment/extra_metrics.csv
Triage queue Hit@k	results/sentiment/triage_queue_hits.csv
Paired sentiment tests	results/sentiment/comparisons.csv
Per-review out-of-fold probabilities	results/sentiment/oof_probabilities.csv
Selective triage	results/sentiment/selective.csv
ROUGE per product	results/summarization/fewsum_by_group.csv
Boundary recovery	results/summarization/segmentation.csv
Telecom digest metrics, 47 clusters	results/summarization/telecom_by_cluster.csv
Generated digests	results/summarization/telecom_digests.csv
SQuAD benchmark	results/qa/squad_reader_benchmark.csv
QA configuration search on development data	results/qa/dev_search.csv
Every held-out QA prediction	results/qa/test_predictions.jsonl
Abstention results	results/qa/test_selective.json
Answer Hit@k, AUROC and error sources	results/qa/qa_extra.json
Integrated outputs and unit tests	results/integrated/demo.json, pytest.txt

B Execution Record

Each notebook's executed run, with its code, cell outputs and log, is public on Kaggle. Each run cloned this repository from GitHub, and the results in this report come from these runs.

Case study	Public Kaggle notebook
Word2Vec	https://www.kaggle.com/code/bharatchandra1234/telelens-01-word2vec
Sentiment Classification	https://www.kaggle.com/code/bharatchandra1234/telelens-02-sentiment
Text Summarization	https://www.kaggle.com/code/bharatchandra1234/telelens-03-summarization
Factoid QA	https://www.kaggle.com/code/bharatchandra1234/telelens-04-factoid-qa
Integrated system	https://www.kaggle.com/code/bharatchandra1234/telelens-05-integrated

Excerpts from the run logs and unit test output:

```
device cuda Tesla T4
9874 reviews 617573 tokens 11434 types
5629 documents 34054 passages 44 dev 88 test 68 answerable test
{'retriever': 'BM25 rerank SGNS 0.25', 'reader': 'RoBERTa-SQuAD2', 'gate': True, 'weight': 2.0}
tau 3.2721939086914062
14 passed (pytest, tests/test_core.py)
```

C Reproducibility

1. Clone https://github.com/BharatChandra-sys/NLP-case-study.

2. Open a notebook from `notebooks` on Kaggle, enable a GPU and internet, and attach the Comcast dataset.
3. Run all cells. The first cell clones the repository and downloads public data; results are written to `results/`.
4. Seeds are fixed at 42. Tables in this report are generated from `results/` by `make_tables.py`.

Reproducibility check. A repeat run from a fresh GitHub clone reproduced the from-scratch SGNS and PA-W2V metrics and the TF–IDF, DistilBERT, summarization and QA headline scores. Two small differences were observed. The gensim baselines train on several threads and are not exactly repeatable: their word-pair metrics varied by about 0.04 between runs, and FastText sentiment macro-F1 moved from 0.863 to 0.857. DistilBERT's 5% deferral count moved from 892 to 895 reviews. No conclusion in this report depends on these differences. Notebooks 01, 03 and 04 were run once more to add the Hit@k and related metrics, and their earlier headline scores did not change. The sentiment metrics in Tables 11, 12 and 13 were calculated from the saved out-of-fold probabilities by `extra_metrics.py`, without retraining.